1. Haga un bloque **`try/catch`** y en su cuerpo llame a una función **`funA`**, a su vez **`funA`** llama a **`funB`**. Declare tanto en **`funA`** como en **`funB`** instancias de un UDT que anuncien su construcción y destrucción. Haga finalmente que **`funB`** haga el **`throw`** de una excepción. ¿Qué puede observar con respecto a las instancias creadas? Coloque múltiples catch para tipos distintos y verifique el correcto funcionamiento cuando dispara excepciones de distinto tipo desde **`funA`**. ¿Qué pasa si dispara una exception de un tipo no "catcheado"? Verifique el comportamiento del **`catch(…)`**.
2. Dado std::vector<std::string> datos = {"12", "abc", "99999999999", "7"};, recorra el vector convirtiendo cada elemento con std::stoi. Sume los válidos y, para cada inválido, imprima un mensaje **distinto** según sea std::invalid_argument o std::out_of_range. El programa no debe terminar ante el primer error.
3. Indique cuáles de las líneas (a) a (d) no compilan y explique por qué.

```
class Cuenta {
    double saldo = 0;
    void registrar(double m) { /* guarda en un historial */ }
public:
    void depositar(double m) { saldo += m; registrar(m); }
    double getSaldo() const { return saldo; }
};

int main() {
    Cuenta c;
    c.depositar(100);         // (a)
    c.saldo = 500;            // (b)
    c.registrar(10);          // (c)
    double s = c.getSaldo();  // (d)
}
```

4. Escriba una clase `Temperatura` que guarde el valor en grados Celsius en un miembro **privado**. Debe tener:

   `bool setCelsius(double c)`: rechaza valores menores a -273.15 devolviendo `false` sin modificar el objeto.
   `double getCelsius() const` y `double getFahrenheit() const`.

   Responda: ¿qué problema aparecería si el atributo fuera público?
5. Compruebe que una clase con constructores privados no puede (por ahora) instanciarse.